Java Scalar User-Defined Function (UDF) examples — temperature-unit conversions

> This package contains **two scalar UDFs** that together illustrate the simplest and oldest category of Flink user-defined function: a pure, row-at-a-time function callable from SQL.
>
> - **`CelsiusToFahrenheit`** — converts a `DOUBLE` Celsius temperature to its Fahrenheit equivalent using `F = (C × 9/5) + 32`.
> - **`FahrenheitToCelsius`** — converts a `DOUBLE` Fahrenheit temperature to its Celsius equivalent using `C = (F − 32) × 5/9`.
>
> Both UDFs ship in the same uber JAR and are registered as separate Flink functions from that same artifact. Together they demonstrate how the `ScalarFunction` API lets you write plain, deterministic Java methods that Flink can call once per input row and inline directly into the query plan — no state, no timers, no operator graph surgery required.

Table of Contents

- **1.0 What is a Scalar UDF?**
  - **1.1 Where scalar UDFs sit in the UDF hierarchy**
  - **1.2 How Flink discovers and calls a scalar UDF**
  - **1.3 Determinism and `NULL` handling**
- **2.0 UDF 1: CelsiusToFahrenheit**
  - **2.1 Conversion logic**
  - **2.2 How it works end-to-end**
- **3.0 UDF 2: FahrenheitToCelsius**
  - **3.1 Conversion logic**
  - **3.2 How it works end-to-end**
- **4.0 Comparing the UDFs in this package**
- **5.0 Resources**

1.0 What is a Scalar UDF?

A **Scalar UDF** is a user-defined function that takes zero or more input values from a single row and returns exactly one value. It is the closest Flink equivalent of a built-in SQL function like `UPPER()`, `ABS()`, or `CAST()`: the function sees one row at a time, has no memory of prior rows, and produces exactly one output value per invocation.

1.1 Where scalar UDFs sit in the UDF hierarchy

Flink exposes four categories of user-defined function. Scalar UDFs are the simplest, and most of the heavier categories build on the same reflection-based `eval()` discovery mechanism:

UDF category	Rows in	Rows out	State	Example use case
Scalar (this example)	1 value from 1 row	1 value	❌	Unit conversion, string cleaning, deterministic math
Table-valued	1 row	0..N rows	❌	Exploding a CSV column into multiple rows
Aggregate	N rows	1 value	✅	`GEOMEAN()`, weighted median
Process Table Function (PTF)	Table argument	0..N rows	✅ (set semantics)	Sessionization, stateful enrichment

Choose a scalar UDF when the computation depends only on the current row's values and can be expressed as a pure function.

1.2 How Flink discovers and calls a scalar UDF

A scalar UDF is a Java class that extends `org.apache.flink.table.functions.ScalarFunction` and declares one or more public methods named `eval`. Flink uses reflection to find the `eval` method whose parameter types match the SQL call site, so:

- The method name **must** be `eval` (lower-case).
- The method **must** be `public`.
- You may overload `eval` with different parameter lists; Flink picks the one that matches the SQL invocation.

Both classes in this package follow the same minimal skeleton:

```java
package scalar_udf;

import org.apache.flink.table.functions.ScalarFunction;

public class CelsiusToFahrenheit extends ScalarFunction {
    public Double eval(Double celsius) {
        if (celsius == null) return null;
        return (celsius * 9.0 / 5.0) + 32.0;
    }
}
```

At SQL level, you register the class as a function that points at the uber JAR on the Flink classpath:

```sql
CREATE FUNCTION celsius_to_fahrenheit
    AS 'scalar_udf.CelsiusToFahrenheit'
    USING JAR 'file:///opt/flink/usrlib/scalar-udf.jar';
```

and call it like any built-in scalar function:

```sql
SELECT sensor_id,
       celsius_temperature,
       celsius_to_fahrenheit(celsius_temperature) AS fahrenheit_temperature
  FROM celsius_reading;
```

1.3 Determinism and `NULL` handling

Both UDFs in this package are **deterministic**: the same input always produces the same output. Determinism is not just documentation — it is a contract that lets the Flink planner safely cache, reorder, and replay calls during failure recovery. If you need a non-deterministic scalar UDF (e.g., one that reads the wall clock), override `isDeterministic()` and return `false`.

Both `eval` methods also preserve SQL `NULL` semantics by returning `null` when the input is `null`. This keeps the UDF transparent to the rest of the query: a `NULL` temperature stays `NULL` through the pipeline rather than silently becoming a numeric surprise.

2.0 UDF 1: CelsiusToFahrenheit

2.1 Conversion logic

Applies the standard formula `F = (C × 9/5) + 32`. The `eval` method takes a nullable `Double` and returns a nullable `Double`, so the function plugs in wherever a `DOUBLE` column flows.

Source: `CelsiusToFahrenheit.java`

2.2 How it works end-to-end

```
Kafka (celsius_reading)
        │
        ▼
  ┌──────────────────┐
  │ celsius_reading  │    Flink SQL source table (JSON / earliest-offset)
  └──────────────────┘
          │
          ▼
  celsius_to_fahrenheit(celsius_temperature)    ← stateless scalar call per row
          │
          ▼
  ┌──────────────────────┐
  │ celsius_to_fahrenheit │    Flink SQL sink table → Kafka (celsius_to_fahrenheit)
  └──────────────────────┘
```

The UDF is invoked once per input row; no keying, no state, no windowing. The Flink planner inlines the call into the scan-and-project pipeline, so throughput is essentially the cost of the arithmetic plus the Kafka I/O.

3.0 UDF 2: FahrenheitToCelsius

3.1 Conversion logic

Applies the inverse formula `C = (F − 32) × 5/9`. Same signature (`Double eval(Double)`), same determinism and `NULL`-preservation guarantees as `CelsiusToFahrenheit` — only the arithmetic differs.

Source: `FahrenheitToCelsius.java`

3.2 How it works end-to-end

```
Kafka (fahrenheit_reading)
        │
        ▼
  ┌──────────────────────┐
  │ fahrenheit_reading   │   Flink SQL source table (JSON / earliest-offset)
  └──────────────────────┘
          │
          ▼
  fahrenheit_to_celsius(fahrenheit_temperature)   ← stateless scalar call per row
          │
          ▼
  ┌──────────────────────┐
  │ fahrenheit_to_celsius │   Flink SQL sink table → Kafka (fahrenheit_to_celsius)
  └──────────────────────┘
```

4.0 Comparing the UDFs in this package

Both classes share the same scalar-UDF contract; the table below highlights the symmetry.

Aspect	`CelsiusToFahrenheit`	`FahrenheitToCelsius`
Base class	`org.apache.flink.table.functions.ScalarFunction`	`org.apache.flink.table.functions.ScalarFunction`
`eval` signature	`Double eval(Double celsius)`	`Double eval(Double fahrenheit)`
Formula	`F = (C × 9/5) + 32`	`C = (F − 32) × 5/9`
State	None	None
Determinism	Deterministic (default)	Deterministic (default)
`NULL` handling	Returns `null` for `null` input	Returns `null` for `null` input
Rows in → rows out	1 → 1	1 → 1
SQL registration	`CREATE FUNCTION celsius_to_fahrenheit AS 'scalar_udf.CelsiusToFahrenheit' USING JAR …`	`CREATE FUNCTION fahrenheit_to_celsius AS 'scalar_udf.FahrenheitToCelsius' USING JAR …`

Both UDFs compile into the same uber JAR (`app-1.0.0-SNAPSHOT.jar`, produced by `./gradlew shadowJar`) and are registered as separate Flink functions via two `CREATE FUNCTION ... USING JAR` statements that point to the same artifact but reference different fully-qualified class names (`scalar_udf.CelsiusToFahrenheit` and `scalar_udf.FahrenheitToCelsius`).

5.0 Resources

- Apache Flink User-defined Functions
- Apache Flink Scalar Functions
- Create a User-Defined Function with Confluent Cloud for Apache Flink